

AI 大模型不难

一本让零基础读者也能笑着看懂 Transformer 的开源电子书

共 29 课 · 五大部分 · 从零到面试通关

目录

开篇

第 0 课 · 第 0 课：开篇，这本书怎么读 + 大模型五分钟全景图

第一部分：世界观 —— 大模型到底是个什么东西

第 1 课 · 第 1 课：Token、预训练、微调，三个词讲完大模型的一生

第 2 课 · 第 2 课：涌现和幻觉，大模型的两个「意料之外」

第 3 课 · 第 3.5 课：解码策略，模型怎么从概率里挑出下一个词

第 3 课 · 第 3 课：你怎么跟模型说话，Prompt + CoT + ICL

第 4 课 · 第 4 课：RLHF 五分钟速览

第 5 课 · 第 5 课：省钱的三个办法，量化、蒸馏、分布式

第二部分：引擎 —— Transformer 里里外外

第 6 课 · 第 6 课：Transformer 到底是什么

第 7 课 · 第 7 课：Attention 公式，手写一遍就懂了

动手算一遍：句子「爱 学习」， $d_k = 2$

第 8 课 · 第 8 课：多头注意力，为什么一个头不够

第 9 课 · 第 9 课：位置编码，模型怎么知道词顺序

第 10 课 · 第 10 课：残差连接，梯度的高速公路

第 11 课 · 第 11 课：LayerNorm，BN 在图像界称王，NLP 为什么不行

第 12 课 · 第 12 课：Pre-norm vs Post-norm，安检放哪的问题

第 13 课 · 第 13 课：两种 Mask，为什么模型必须戴眼镜

第三部分：炼金 —— 模型是怎么训出来的

第 14 课 · 第 14 课：参数 vs 数据，容器和内容

第 15 课 · 第 15 课：MLM vs CLM，两种截然不同的教法

第 16 课 · 第 16 课：RLHF 全流程，SFT → RM → PPO

第 17 课 · 第 17 课：DPO 和 GRPO，更聪明的对齐方式

第 18 课 · 第 18 课：LoRA，穷玩大模型的终极答案

动手算一遍：一个 4×4 的权重矩阵， $r=2$

第 19 课 · 第 19 课：训练硬核补充，过拟合、梯度、Warmup、ZeRO

第四部分：落地 —— 大模型的十八般武艺

第 20 课 · 第 20 课：RAG，让模型会查资料

第 21 课 · 第 21 课：推理优化全家桶

第 22 课 · 第 22 课：AI Agent，让模型会干活

第 23 课 · 第 23 课：MCP / A2A / Function Call，让模型会叫外援

第 24 课 · 第 24.5 课：多模态/VLM 入门，模型怎么看图

第 24 课 · 第 24 课：前沿架构，MoE、GQA、DeepSeek 怎么玩的

第五部分：实战 —— 面试现场还原

第 25 课 · 第 25 课：大厂真题精讲（上）

第 26 课 · 第 26 课：大厂真题精讲（下）+ 系统设计 + 项目讲述

第 0 课：开篇，这本书怎么读 + 大模型五分钟全景图

三个月前我准备 AI 技术面，背了一沓面试八股，背到第三天就崩溃了。不是因为记不住。是面试官每个词都能追着问，LoRA 原理？跟 QLoRA 的区别？为什么不用 Adapter？

后来我发现，大模型里那些让人头大的概念，每个都对应一个生活里的比喻。Attention 是图书馆查资料。残差连接是传话游戏的复印件。LayerNorm 是把全班身高统一成标准尺。

这本书就是把这些比喻整理了出来。26 课，分五部分。

第一部分，世界观（1-5 课）：Token、预训练、微调，涌现和幻觉，Prompt 工程，RLHF，量化蒸馏分布式。

第二部分，引擎（6-13 课）：Transformer 架构，Attention 公式，多头注意力，位置编码，残差，归一化，Pre/Post-norm，两种 Mask。面试里被问得最多的八课。

第三部分，炼金（14-19 课）：参数 vs 数据，MLM vs CLM，RLHF 全流程，DPO/GRPO，LoRA，训练硬核。

第四部分，落地（20-24 课）：RAG，推理优化，Agent，MCP 协议，前沿架构。

第五部分，实战（25-26 课）：十道大厂真题 + 系统设计。

零基础顺着读。面试突击跳到 25-26 课刷真题。有基础的跳着当词典查。

AI 大模型世界地图

26 课，五站旅程——从零到面试通关

开篇

第 0 课 · 地图

第一部分：世界观 —— 大模型到底是个什么东西

第 1-5 课：Token · 涌现与幻觉 · Prompt/CoT/ICL · RLHF 速览 · 量化/蒸馏/分布式

第二部分：引擎 —— Transformer 里里外外

第 6-13 课：Transformer · Attention 手写 · 多头注意力 · 位置编码 · 残差连接 · LayerNorm · Pre/Post-norm · 两种 Mask

第三部分：炼金 —— 模型是怎么训出来的

第 14-19 课：参数量/数据量 · MLM/CLM · RLHF 全流程 · DPO/GRPO · LoRA · 训练硬核

第四部分：落地 —— 大模型的十八般武艺

第 20-24 课：RAG · 推理优化 · Agent · MCP/A2A · 前沿架构(MoE/DeepSeek)

实战

第 25-26 课 · 大厂面试

AI 大模型世界地图

磨平一些信息差。

第 1 课：Token、预训练、微调，三个词讲完大模型的一生

Token — 模型的字母表

模型不直接读汉字或英文单词。它读的是 Token，一种介于「字符」和「词」之间的最小处理单元。

Tokenization 的主流算法是 BPE (Byte Pair Encoding)。它从字符起步，反复合并最常相邻出现的字符对，逐步构建词表。「我喜欢」可能先合并「喜」+「欢」→「喜欢」，再合并「我」+「喜欢」→「我喜欢」。高频词合并成大块，生僻字保持碎片。GPT-4 的词表约 10 万个 Token，全世界的语言都在这 10 万块积木里拼。

一个 Token 约等于 0.75 个英文单词，或 0.5 个汉字。「我要一杯珍珠奶茶」约 6 个 Token，高频词组「珍珠奶茶」作为一个整体占 1 个 Token。

Token 进了模型之后，被映射成一个高维向量 (Embedding)，比如 768 维。这个向量不是人为定义的，是训练过程中自己学会的。意义相近的词向量距离近，「猫」和「狗」的向量夹角比「猫」和「汽车」小得多。

预训练 — 猜了几十万亿次的下一个词

给了模型几十万亿 Token 的互联网文本。训练方法极其简单：让它猜下一个词。

「今天天气怎么」→ 模型预测下一个 Token 是「样」。猜对了，参数不动。猜错了，算 Loss (交叉熵损失，衡量预测分布和正确答案之间的差距)，反向传播，梯度下降更新参数，再去猜下一个。

每猜错一次就微调一次参数。几十万亿次之后，模型从随机噪声变成了一个能组织语言的大脑。它学会了语法 (形容词修饰名词)、常识 (下雨要带伞)、甚至一些推理 (如果 $A > B$ 且 $B > C$ ，那么 $A > C$)。没有人教它语法规则，它从几十万亿次「猜词-纠错」的循环里自己归纳出来的。

这几十万亿次计算的总成本：GPT-4 级别约 1 亿美元的电费和 GPU 租金。这也是为什么只有少数公司能训基座模型。

微调 — 通用大脑做特定训练

预训练完的模型什么都会一点，但什么都不精。你问它客服问题，它可能回答得像散文。微调就是在特定领域的高质量数据上再做几轮猜词训练。

准备 5000 条奶茶店客服对话：「珍珠奶茶有中杯和大杯吗」「有，中杯 15 元大杯 20 元」，让模型在这些数据上继续猜下一个词。参数从预训练的「通用语言」往「奶茶店客服」方向微调。

底层的语言能力还在（语法、常识），但输出风格、知识侧重、行为模式全调整了。LoRA 微调只改不到 1% 的参数就能做到，第 18 课详讲。

走一遍完整流程：奶茶店客服机器人

Step 1, Tokenize: 「我要一杯珍珠奶茶少糖」→ BPE 切成 [我要, 一杯, 珍珠, 奶茶, 少, 糖]。6 个 Token。

Step 2, 预训练模型处理: 每个 Token 映射成 768 维 Embedding → 过 32 层 Transformer → 输出下一个 Token 的概率分布。它知道「珍珠奶茶」是饮品、「少糖」是甜度选项。但它不知道你们店的菜单。概率前三是「好的请稍等」「加珍珠吗」「建议搭配炸鸡」。

Step 3, 微调后: 反复在奶茶店真实对话上训了几千步。模型不再提炸鸡，它学会了推销第二杯半价、确认杯型和温度、告知取餐时间。只改了不到 1% 的参数，行为全变了。

Token 是原料，预训练是上学，微调是实习。

磨平一些信息差。

第 2 课：涌现和幻觉，大模型的两个「意料之外」

GPT-3 在 130 亿参数时不会加法，650 亿也不会，1300 亿也不会。到了 1750 亿，突然会了。不是从 60 分涨到 80 分，是从 20 分跳到 80 分。

涌现：为什么参数堆到某个量级，能力会跳变

这跟 Scaling Law 直接相关。第 14 课会讲公式，这里先讲直觉。

Loss 随参数量 N 和数据量 D 的增长是平滑的幂律下降： $L \propto 1/N^\alpha + 1/D^\beta$ 。但 Loss 是一个全局指标，它衡量模型在所有 token 上的平均预测准确率。具体到某个任务（比如三位数加法），Loss 从 4.0 降到 3.5 的阶段可能准确率一直停在 20%。但当 Loss 降到某个临界点（比如 3.2 以下），模型突然「学会」了加法，准确率跳到 80%。

Loss 是平滑的，但能力是突变的。因为任务本身有一个「最小所需精度」阈值。就像打靶：靶子 10 环，你从脱靶练到 3 环再练到 6 环都是不及格（没命中靶心）。但一旦突破到 7 环以上，开始稳定命中，在外人看来就是「突然会了」。

涌现不是魔法，是平滑优化曲线在离散评测指标上的相位突变。和物理里的相变完全一个道理：水从 99°C 到 100°C 的温度变化是线性的，但物态从液态到气态是跳跃的。

幻觉：猜词机器为什么会「编造」

模型就是一个猜下一个词的概率机。给定 prompt 「推荐三本明朝历史的书」，它开始逐 token 生成。生成第一个词时，它在 10 万 Token 词表上输出概率分布。输出「《」的概率最高，输出「万」的概率也差不多，输出「明」的概率排第三。

注意：它不是在「检索知识」。它是在计算给定上下文后，哪个 Token 出现的条件概率最高。它看到的训练数据里有无数「书名 = 《XXXX》」的模式。所以它顺着这个模式往下猜，《大明》《明朝》《明史》...逐步拼出一串看起来像书名的 Token。

当它不认识任何真正的明朝历史书时，它不会停。条件概率仍然存在，「万历十五年」出现在「明朝」「历史」「推荐」这些上下文里的概率比「香蕉飞船」高得多。所以它输出了一本不存在的书，但书名的每个字在统计意义上都是「合理的」。

RLHF 让问题更微妙。RM 训出来之后，模型学到一个规律：「给出确定答案」的回答得分比「我不知道」高。于是面对不确定性，模型倾向于猜一个最高概率的 Token 序列而不是承认无知。

正确防御：给 prompt 加「如果信息不确定，请直接说明不知道，不要猜测」。这句话改变了条件概率分布，「我不知道」这个 Token 序列的概率被抬高，模型更容易选择承认无知。

走一遍幻觉是怎么产生的

你问：「《大秦帝国》孙皓晖写的，总共有几卷？」

模型训练数据里见过「大秦帝国」「孙皓晖」「历史小说」共现，但没记住确切卷数。它不输出「我不知道」，因为 RLHF 训出的行为是「尽量给答案」。

它开始猜词：历史小说通常是三到六卷，大秦帝国这种大制作更可能五到六卷。逐 Token 生成「六」，然后继续生成「卷」，然后为显得专业，再加「分别是」然后编出了六卷的名字。每一步的 Token 在统计上都是合理的，只是它们的组合对应了一本不存在的书。

磨平一些信息差。

第 3.5 课：解码策略，模型怎么从概率里挑出下一个词

Attention 算完了，Transformer 跑完了，Softmax 输出了一个概率分布：[0.02, 0.15, 0.45, 0.08, 0.30]。总共 10 万个候选 Token，每个有个概率。现在要挑一个出来当下一个词。

怎么挑？五种挑法。

Greedy Search — 永远选概率最大的

每步挑概率最高的 Token。快，确定性强，每次同样的 prompt 永远输出同样的结果。致命伤：单调重复。因为一旦第一步选了「喜欢」，下一步「喜欢」之后最高概率的又是「你」，然后「喜欢」「你」「喜欢」「你」无限循环。不做任何创意生成。

面试时说：Greedy 适合翻译、代码补全这类确定性任务。不适合开放式写作和对话。

Beam Search — 保留K条候选路

不只保留最优的一条路，而是保留 K 条。K=4 就是 4 条。每一步对每条路径的所有可能 Token 做扩展，算累积概率，砍掉不行的，保留 Top-K。

质量比 Greedy 高得多，「我昨天去了动物园，看到了一头大象」的概率比「我喜欢你喜欢你」高几百倍，Beam Search 能找出来。代价是 K 倍计算量，K 越大越贵。

面试时说：Beam Search 是翻译和摘要的标配。但 K 太大 (>10) 边际收益递减，K=3-5 最实用。

Top-K Sampling — 从Top-K里随机抽

不选固定的，从概率最高的 K 个 Token 里随机抽一个。K=50 就是从 Top-50 里按概率权重随机选。

引入随机性解决了重复循环。但如果 Top-50 里有一堆低概率的奇怪 Token 混进来，可能偶尔输出离谱词。

面试时说：K 是静态的，不随分布变化。一个很集中的分布（第一个词 0.9，其余 0.1），K=50 会把大量低概率 Token 纳入候选。一个很分散的分布（所有词都差不多），K=50 可

能砍掉合理候选。

Top-P (Nucleus) Sampling — 累积概率超过P就停

自适应版 Top-K。从概率最高的 Token 开始累加，累到超过 P（通常 0.9）就停，只在这个最小集合里随机抽。

分布集中时，「the」概率 0.85，「a」0.08，Top-P 只需要 2 个 Token 就超过 0.9。分布分散时，20 个 Token 各有 0.04-0.06 概率，Top-P 会纳入所有 20 个。

GPT 系列的默认策略就是 Top-P，P 通常 0.9-0.95。

面试时说：Top-P 比 Top-K 更鲁棒，因为它随分布自动调整候选集大小。实际工程中两者常混用：先 Top-K 砍到 50，再 Top-P 过滤。

Temperature — 控制概率分布的平滑度

在 Softmax 之前，把 logits 除以温度 T：

$$\text{softmax}(\text{logits} / T)$$

T=1：原始分布，不改。T=0.5：分布更尖锐，高概率更高、低概率更低，Greedy 和 Sampling 都更确定。T=2.0：分布更平滑，各 Token 概率差距缩小，Sampling 更随机多样。

T→0 逼近 Greedy。T→∞ 变成完全随机采样。标准区间：T=0.7-1.0 适合对话，T=0.2-0.5 适合代码/翻译，T>1.0 适合创意写作。

面试时说：Temperature 是控制「创造性 vs 确定性」的最直观旋钮。面试官问你调过什么参数来改善输出质量，Temperature 是最安全、最管用的答案。

一句话选型

任务	策略	温度
翻译、代码	Beam Search (K=3-5)	0.2-0.5
对话、客服	Top-P (0.9)	0.7-0.8

任务	策略	温度
创意写作、头脑风暴	Top-P (0.95) + Top-K (50)	1.0-1.2
数学推理	Greedy + CoT	0.0-0.1

磨平一些信息差。

第 3 课：你怎么跟模型说话，Prompt + CoT + ICL

Prompt Engineering 的核心就一件事：把要求写具体。「写个方案」不行，「写一个针对大学生的校园奶茶店营销方案，预算 5000 元，分线上线下各三项措施」才行。

记忆框架 CRISP。Context 上下文，你是谁什么场景。Role 角色，你是一个毒舌美食评论家。Instruction 指令，写 300 字分三段。Style 风格，用初中生能看懂的语言。Purpose 目的，这份材料要给投资人看的。五要素配齐，输出质量直接上一个台阶。

CoT — 为什么「一步步思考」真的有用

不是魔法。是自回归生成的自然结果。

模型一次只生成一个 Token。没有 CoT 时，它面对「小明分糖果还剩几颗」这个问题，必须在第一个输出 Token 就给出最终答案。这对一个概率模型来说，是把多步推理压缩到一步，中间每一步都可能算错。

有了 CoT，提示「我们一步步来」。模型的第一个输出变成「步骤 1：小明最初有 15 颗糖」。这个输出立即被追加到对话历史里，作为下一次生成的上下文。所以生成「步骤 2」的时候，模型不仅能看到原始题目，还能看到自己刚生成的「步骤 1」的结果。

CoT 的本质是让模型的输出变成自己的输入。它自己给了自己额外的推理锚点。每一步的正确结果通过自回归机制自然传导到下一步，不是模型变聪明了，是它的工作记忆变大了。

这和人类做复杂数学题要打草稿是同一个道理。不是你的大脑在草稿纸上变聪明了，是你把中间结果写了下来，下一步不用重新算。

ICL — 不训练就学会新任务

In-Context Learning。在 prompt 里贴 2-3 个例子，模型看完就能推断任务模式，零参数更新。

ICL 的机制：贴进去的例子经过 Transformer 的前向传播后，它们的 KV 向量留在了注意力层里。当模型处理真正的 query 时，这些「例子」的 KV 向量可以通过自注意力被查询，

模型从例子中提取任务模式，直接应用到当前 query 上。

不是「学习」。是「用注意力在上下文中找到了最相似的模板然后跟着做」。参数一个都没动。整个过程就是一次前向传播。

例子质量和多样性决定效果。贴三个例子都是一种模式，模型对第四种模式就懵。贴三个不同模式但每个都清晰标注，模型能同时学会四种。

CoT 是怎么生效的，一个数学题走一遍

题目：「小明有 15 颗糖，给了小红 3 颗，又买了 5 颗，然后分给 4 个同学每人一样多，还剩几颗？」

不用 CoT，模型直接从 prompt 生成答案，它得在一个输出里算出结果。经常错。

用 CoT，加一句「我们一步步算」：

步骤 1：小明最初有 15 颗糖
步骤 2：给小红 3 颗， $15 - 3 = 12$ 颗
步骤 3：又买 5 颗， $12 + 5 = 17$ 颗
步骤 4：分给 4 个同学， $17 \div 4 = 4$ 剩 1
答案：1 颗

模型生成步骤 1 时，只看原始题目。生成步骤 2 时，看原始题目 + 步骤 1 作为上下文。生成步骤 4 时，上下文里已经有了前 3 步的正确结果。CoT 把一个大推理拆成了 4 个小推理链，每一步的正确答案通过自回归自然流入下一步。

磨平一些信息差。

第 4 课：RLHF 五分钟速览

GPT-3 会说人话但不知道什么该说什么不该说。问它讲故事它会，问它做炸弹它也给步骤。不是邪恶，是没有「好坏」概念。

RLHF 三步，像训练脱口秀演员。

第一步 SFT，请家教。人类标注员手写几千条「完美回答」，模型照着学。学完能从「会说完整句子」变成「能写及格段子」。

第二步 RM，请评委。标注员给同一个问题的多个回答排序，A 比 B 好，C 比 D 差。训练一个打分模型，学会判断回答的质量。

第三步 PPO，反复彩排。模型生成回答，RM 打分，根据分数调策略。高分多保留，低分少用。一万遍后模型能稳定输出高质量回答。

用一句话走通整个 RLHF 流程

Prompt：「推荐一本入门 Python 的书」

原始预训练模型可能答什么：从 Reddit 学到「买我的课」「扫码加群」这种垃圾回复。因为它见过的网页里这种文案太多了。

SFT 阶段：人类标注员写了 50 条「理想回答」范例，推荐《Python 编程从入门到实践》，说明适合零基础、有练习题、第三版更新到 Python 3.11。SFT 训练后，模型学会了回答应该「具体、有用、不卖课」。

RM 阶段：给 SFT 模型同一个 Prompt，让它生成 4 条回答。标注员排序：

- A（五星）：推荐一本经典入门书 + 讲清楚为什么适合 + 附学习路径
- B（四星）：推荐一本书 + 简单介绍
- C（两星）：列了五本书但没说明哪本好
- D（零星）：「买我的课，7 天学会 Python」

RM 学会给 A 打高分、D 打零分。

PPO 阶段：模型生成回答 → RM 打分 → 反向传播调参数。生成 D 类的回答概率被压低，生成 A 类的被抬高。一万轮后模型不再输出卖课文案。

磨平一些信息差。

第 5 课：省钱的三个办法，量化、蒸馏、分布式

量化，把 32 位参数压到 8 位或 4 位。FP32→INT8 显存砍四分之三，28GB 变 7GB。INT4 再砍一半到 3.5GB。核心假设：参数里大部分精度是冗余的。

蒸馏 — 大模型教小模型，不是抄答案，是学思路

蒸馏不是让小模型直接背大模型的输出。它利用大模型输出的「软标签」，不是「答案是 B」这种硬标签，而是「A 的概率 2%，B 的概率 85%，C 的概率 10%，D 的概率 3%」这种完整概率分布。

这些概率分布里包含了大模型的「知识」：虽然正确答案是 B，但 C 也有 10% 的概率，说明 C 和 B 有一定相关性。小模型学习这个分布，相比只学「答案是 B」，能获得更丰富的训练信号。

蒸馏的损失函数：

$$\text{Loss} = \alpha \cdot \text{KL}(p_{\text{soft}} \parallel q) + (1-\alpha) \cdot \text{CrossEntropy}(y_{\text{true}}, q)$$

两个来源的信号混合。KL 散度那项让小模型的输出分布逼近大模型的软标签，CrossEntropy 那项确保不能偏离真实答案。 α 是混合系数，通常 0.7-0.9。温度 T 是另一个关键参数，软标签在送入 KL 前先除以 T（通常 2-5），让分布更平滑。温度越高，大模型给的「C 也有 10% 概率」这类副信息越明显，小模型学到的细节越多。

蒸馏出来的 7B 小模型能达到 70B 大模型 80%-90% 的水平，但推理成本只有几分之一。手机上跑的那些 AI 助手、输入法预测，全是蒸馏出来的。

分布式训练 — GPU 之间怎么传消息

数据并行：4 张 GPU，batch 切成 4 份，各算各的梯度。算完后通过 all-reduce 通信原语汇总梯度：每个 GPU 把自己算的梯度广播给所有其他 GPU，同时接收别人的，最终每个 GPU 都拿到 4 份梯度的平均值。用这个平均值更新参数，保证 4 张卡的参数完全一致。加速训练时间，但每张卡都要存完整的模型 + 优化器状态，不省显存。

all-reduce 的通信量随 GPU 数量线性增长。8 卡还行，128 卡通信开销可能超过计算时间。梯度累积是缓解方案：多次小 batch 算梯度，攒够 N 次再 all-reduce，减少通信频次。

模型并行：模型拆成两半，前半放 GPU 0，后半放 GPU 1。数据先过 GPU 0 的前半段，输出传给 GPU 1 继续算。只有一张卡在干活的时候另一张在等，这叫 pipeline bubble（流水线气泡），GPU 利用率有天然的损失。优势是省显存，每张卡只存一半模型。

实际训练三者混合：数据并行跨 8 台机器，每台机器内部 8 张卡做模型并行，再加 ZeRO 优化器状态分片（第 19 课详讲）。千卡级别的训练就是这么搭起来的。

量化到底损失了多少精度，一个参数走一遍

拿一个注意力层的权重矩阵，里面有个值是 0.718382（FP32，32 位浮点数）。

FP16 量化：0.718382 → 0.71875。误差 0.0004，肉眼不可见。

INT8 量化：0.718382 先缩放到 [-127, 127] 范围。这一层最大值 1.5，最小值 -1.5， $(0.718382 / 1.5) \times 127 \approx 60.8 \approx 61$ 。解码回来 $61 / 127 \times 1.5 \approx 0.72047$ 。误差 0.002，基本不影响结果。

INT4 量化：范围只有 [-7, 7]。 $(0.718382 / 1.5) \times 7 \approx 3.35 \approx 3$ 。解码回来 $3 / 7 \times 1.5 \approx 0.6429$ 。误差 0.075，数学推理崩了。但 QLoRA 在量化后的模型上再加 LoRA 微调，训练过程中能把这个误差补回来。

量化是减肥，蒸馏是拜师，分布式是喊人搬砖。三种方法经常一起上：先蒸馏成小模型，再 INT4 量化，分布式部署到手机端。

磨平一些信息差。

第 6 课：Transformer 到底是什么

2017 年 Google 发了篇论文，《Attention Is All You Need》。没吹牛。GPT、Claude、Llama、DeepSeek 全是它的直系后代。

要理解 Transformer 为什么是革命，得先知道它革了谁的命。

RNN：一条路走到黑

在 Transformer 之前，处理文本的主流方案是 RNN（循环神经网络）。

RNN 的核心操作极其简单：它一次只读一个词，读完更新自己的「记忆」，再读下一个。数学上就是一个循环公式：

$$h_t = f(W \cdot h_{t-1} + U \cdot x_t)$$

h_t 是读完第 t 个词之后的记忆状态。 h_{t-1} 是前一步的记忆。 x_t 是当前词。 W 和 U 是可训练的权重矩阵。 f 是激活函数。每读一个词，就用当前词 + 上一步记忆，算出新的记忆。

读「猫坐在垫子上」的过程：

t=1: 读「猫」	$\rightarrow h_1 = f(W \cdot h_0 + U \cdot \text{「猫」})$	记忆里只有「猫」
t=2: 读「坐」	$\rightarrow h_2 = f(W \cdot h_1 + U \cdot \text{「坐」})$	记忆里有「猫」和「坐」
t=3: 读「在」	$\rightarrow h_3 = f(W \cdot h_2 + U \cdot \text{「在」})$	逐步积累
t=4: 读「垫子」	$\rightarrow h_4 = f(W \cdot h_3 + U \cdot \text{「垫子」})$	
t=5: 读「上」	$\rightarrow h_5 = f(W \cdot h_4 + U \cdot \text{「上」})$	最终状态包含整句话

两个致命问题。第一，串行。 h_5 必须等 h_4 算完， h_4 等 h_3 。5 个词要算 5 步，GPU 大部分时间在等，不能并行。第二，长距离遗忘。 h_t 是通过 W 反复相乘传下来的。 W 的值通常略小于 1，连乘 100 次之后，100 步前的信息几乎消失。「猫」这个词的信息传到「垫子」时已经稀释了几百倍。

LSTM 和 GRU 加了「门控」机制缓解遗忘，核心是一套「记住什么、忘掉什么」的选择开关。长距离记住了，但串行的本质没变。

Transformer：所有词同时互相看

Transformer 的核心操作不再是按顺序读。它的自注意力机制一次性把所有词扔进一个计算图，让每个词同时「查询」所有其他词。

$$\text{Attention}(Q,K,V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

不需要等上一步。不需要通过 W 反复相乘传递信息。每个词通过 QK^T 矩阵乘法直接跟所有其他词算相关性。第 1 个词和最后 1 个词之间的「距离」不是 100 步递推，而是一次点积。长度 5 还是 50000，一次矩阵乘法全解决。这在第 7 课会拆开讲。

两个字面叫「自注意力」，Q、K、V 都来自同一份输入数据，是「自己看自己」。

Encoder — 一眼看完全文

Encoder 的任务是把输入句子变成一串「理解过的」向量。输入 5 个 Token，输出 5 个等长的向量，每个向量里融合了整句话的上下文信息。

它用双向自注意力：处理「猫」这个位置时，不光看前面的「猫」，也看后面的「垫子」「上」。「猫」的最终向量是整句话所有词的加权混合。

为什么需要双向？因为理解「猫坐在垫子上」这句话里的「猫」，上下文信息都重要。「猫」是主语（后面的「坐」告诉你的），「猫」是什么（前面的定冠词或修饰语告诉你的）。只看一边，理解是不完整的。

BERT 只用了 Encoder。因为分类、命名实体识别、问答理解这些任务不需要生成新文本，只需要把输入「读懂」。

Decoder — 一个词一个词往外蹦

Decoder 的任务是生成。输入 5 个 Token（或是 Encoder 的输出），逐个位置预测下一个词。

它的自注意力是因果的，加了下三角 Mask。生成第 3 个词时只能看到第 1、第 2 个词，不能看第 4、第 5。因为推理时第 4 个词还不存在。

GPT 只用了 Decoder。去掉 Encoder，去掉 Cross-Attention，只保留因果自注意力。输入 prompt 「今天天气」，逐 Token 预测下一个词，把生成的词再喂回去当输入，这就是自回归生成。

只用 Decoder 能「理解」prompt 吗？能。因果自注意力虽然不能看未来，但能看全部过去。你的 prompt 就是它的过去。读到「猫坐在」这三个字要生成下一个词时，「猫」「坐」「在」都在它的注意力范围里。

Cross-Attention — Decoder 向 Encoder 要信息

Encoder 已经「读懂了」输入。Decoder 在生成时除了看自己已生成的词，还要查 Encoder 的输出。这个机制叫 Cross-Attention。

Cross-Attention 和自注意力的公式完全一样，区别是 Q、K、V 的来源不同：

	Q（我要查什么）	K、V（被查的资料）
自注意力	来自同一份输入	来自同一份输入
Cross-Attention	来自 Decoder 当前位置	来自 Encoder 的全部输出

Decoder 在生成「like」这个词时，用自己的 Query 去查 Encoder 输出的 Key，发现「喜欢」这个位置最相关，就从 Encoder 的 Value 中提取「喜欢」的语义。翻译场景最直观：Encoder 读中文，Decoder 写英文，Cross-Attention 就是查字典。

T5 是 Encoder-Decoder 完整版。机器翻译、文本摘要这种输入和输出不是同一种「语言」的任务，用完整版最自然。

三种变体一句话

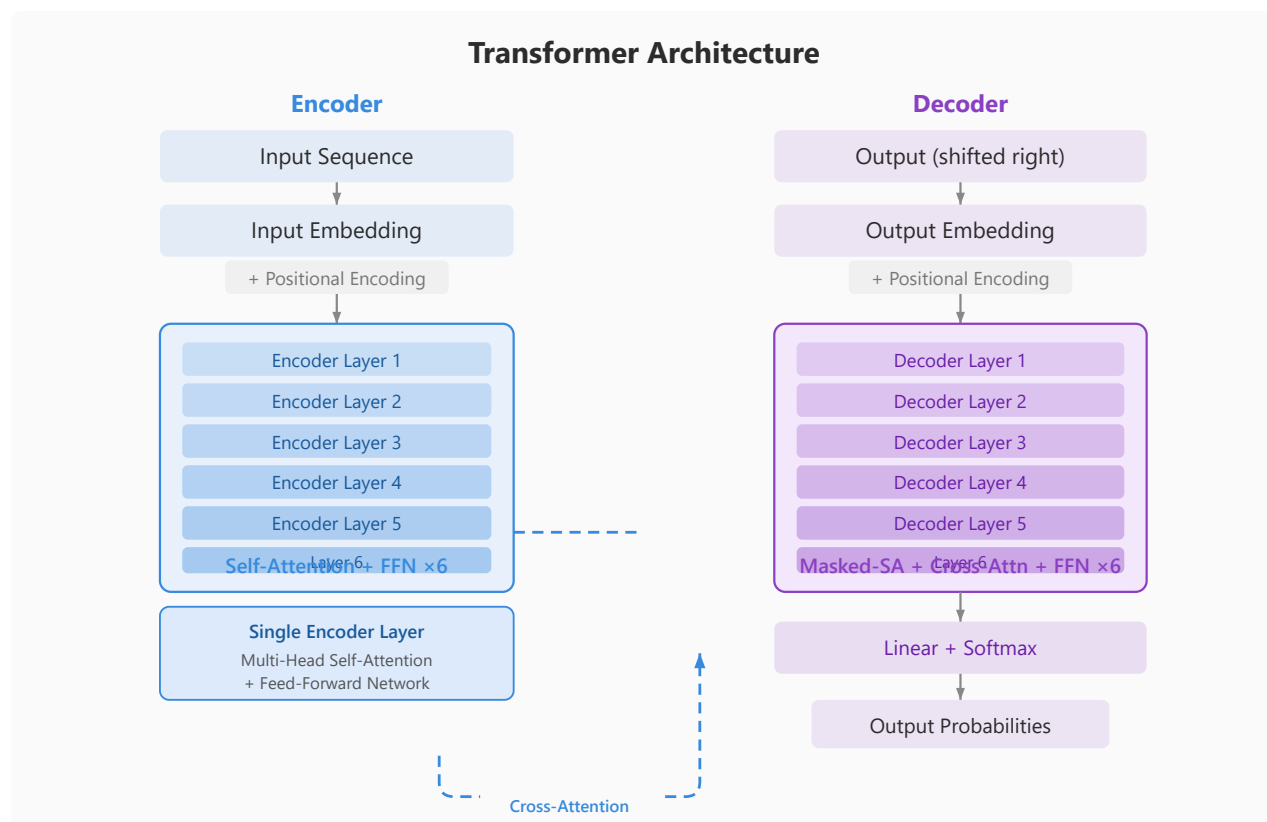
模型	用了什么	适合什么
BERT	只有 Encoder	理解：分类、NER、问答
GPT	只有 Decoder	生成：对话、写作、代码
T5	Encoder + Decoder	转换：翻译、摘要

共享的砖块只有四种

不管怎么组合，Encoder 还是 Decoder，每层里面反反复复就用这四种零件：

注意力机制 — 每个词跟所有其他词对话。残差连接 — 梯度高速公路，防深层训不动。层归一化 — 统一数值范围，防爆炸和消失。位置编码 — 注入词序，不然模型分不清「猫追老鼠」和「老鼠追猫」。

GPT-4 堆了几百层，每层的砖还是这四种。堆得多，能力就大。就这么简单。



Transformer 架构图解

走一遍：一句话穿过 GPT 的完整路径

输入：「猫坐在垫子上」

Step 1 — Tokenize + Embedding：切成 5 个 Token，每个映射成 768 维向量。

Step 2 — 位置编码：位置 0 的「猫」旋转一个角度，位置 1 的「坐」转更大角度。现在每个向量知道自己排第几。

Step 3 — Attention 层（第 1 层）：「垫子」通过 Attention 发现自己跟「猫」「坐」关系最紧，跟「上」的关系没那么紧。所有 Token 互相看，加权融合信息。

Step 4 — 残差 + LayerNorm：加回原始输入，归一化。

Step 5 — FFN 层：每个 Token 独立过两层全连接，做一次非线性变换。

Step 6 — 重复 32 层：每层都在上一步基础上提取更高级的语义关系。第 1 层可能学到「垫子」跟「猫」相关，第 20 层可能学到整句话的动作逻辑。

Step 7 — 输出：最后一个 Token 的向量过 Linear + Softmax，从 10 万 Token 词表里挑概率最高的那个，输出下一个词。

磨平一些信息差。

第 7 课：Attention 公式，手写一遍就懂了

Attention 像在图书馆查资料。Q 是你的问题，K 是每本书的标签，V 是书的内容。

公式四步。

第一步， $Q \times K^T$ 。计算你的问题和每本书标签的相似度。结果是 $n \times n$ 矩阵，每个 token 对每个 token 的相关性分数。这就是 $O(n^2)$ 的来源。

第二步，除以 $\sqrt{d_k}$ 。Q 和 K 的维度大了点积值会爆炸，送进 softmax 后梯度消失。除以 $\sqrt{d_k}$ 把数值压回稳定区间。不除的话训练会发散。

第三步，softmax。把分数变成概率，每一行加起来等于 1。 e^x 让高分更突出、低分接近零。但 $e^0 = 1$ ，所以即使两个词完全正交，注意力也不会是零，每个词都能分到一点关注。

第四步， $\times V$ 。按 softmax 算出的权重对 V 做加权求和。「猫坐在垫子上」里，「坐」的输出 = $V_{\text{坐}} \times 0.5 + V_{\text{猫}} \times 0.3 + V_{\text{垫子}} \times 0.15 + V_{\text{上}} \times 0.05$ 。它不是「坐」的原始含义，而是融入了上下文的新表示。

$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) V$

就这一行。四行 PyTorch 写完。

```
def attention(Q, K, V):
    d_k = Q.size(-1)
    scores = Q @ K.transpose(-2, -1) / (d_k ** 0.5)
    return F.softmax(scores, dim=-1) @ V
```

动手算一遍：句子「爱 学习」， $d_k = 2$

设 Q、K、V 如下（故意设得极简，方便手算）：

$$Q = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \quad K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \quad V = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}$$

爱 学习 爱 学习 爱 学习

Step 1: $Q \times K^T$

$$\begin{aligned} [1,0] \cdot [1,0]^T &= 1 & [1,0] \cdot [0,1]^T &= 0 \\ [0,1] \cdot [1,0]^T &= 0 & [0,1] \cdot [0,1]^T &= 1 \end{aligned}$$

$$QK^T = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

「爱」和「学习」正交，交叉为 0。只有自己跟自己相关。

Step 2: $/ \sqrt{2} \approx / 1.414$

$$\begin{bmatrix} 0.71 & 0 \\ 0 & 0.71 \end{bmatrix}$$

Step 3: Softmax (按行)

第一行「爱」: $e^{0.71}=2.03, e^0=1, \text{sum}=3.03 \rightarrow [2.03/3.03, 1/3.03] = [0.67, 0.33]$

第二行「学习」: $e^0=1, e^{0.71}=2.03 \rightarrow [1/3.03, 2.03/3.03] = [0.33, 0.67]$

$$\text{Attention Weights} = \begin{bmatrix} 0.67 & 0.33 \\ 0.33 & 0.67 \end{bmatrix}$$

注意：原始交叉为 0，但 $e^0=1$ ，所以不是零。注意力有保底。

Step 4: $\times V$

$$\begin{aligned}\text{Output}_{\text{爱}} &= 0.67 \times [2, 0] + 0.33 \times [0, 2] = [1.34, 0.66] \\ \text{Output}_{\text{学习}} &= 0.33 \times [2, 0] + 0.67 \times [0, 2] = [0.66, 1.34]\end{aligned}$$

「爱」的输出不只是自己的 $[2, 0]$ ，混入了 33% 的「学习」。「学习」的输出不只自己的 $[0, 2]$ ，混入了 33% 的「爱」。

这就是 Attention 的核心：每个 token 的输出是所有 token 的加权混合。正交不代表绝缘，softmax 的保底机制让信息永远在流动。

磨平一些信息差。

第 8 课：多头注意力，为什么一个头不够

上节课讲了 Attention 的核心公式，Q 和 K 点积得到一个标量，表示两个词的相关性。但语言里同时存在几十种关系，主谓、动宾、指代、修饰、时间、地点，如果只靠一个标量来描述「猫」和「坐」之间的所有关系，结果只能是模糊的平均值。

多头的数学机制

单头注意力：输入 X 的维度是 512，通过三个矩阵 W_Q 、 W_K 、 W_V （都是 512×512 ）各投影一次，得到 Q、K、V 都是 512 维。然后做 QK^T ，得到 $n \times n$ 的注意力矩阵。

多头注意力：不做 512 维的全量投影。而是把 Q、K、V 各自切成 8 份，每份 64 维。

$$Q = [Q_1, Q_2, \dots, Q_8], \quad K = [K_1, K_2, \dots, K_8], \quad V = [V_1, V_2, \dots, V_8]$$

每个头在自己的 64 维子空间里独立运行一套完整的 Attention：

$$\text{head}_i = \text{Attention}(Q_i, K_i, V_i) = \text{softmax}\left(\frac{Q_i K_i^T}{\sqrt{64}}\right) V_i$$

8 个头的输出拼接起来：

$$\text{MultiHead} = \text{Concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_8) \cdot W_O$$

Concat 的结果是 $8 \times 64 = 512$ 维，和单头一样大。最后用一个 512×512 的 W_O 矩阵融合 8 个视角。

为什么 64 维够用，512 维反而不好

注意力分数 $Q_i \cdot K_j$ 是一个标量。不管你用 64 维还是 512 维，两个词之间最终只输出一个数字。1 个头 = 1 个数字描述所有关系，8 个头 = 8 个数字各描述一种关系。

核心不在于维度大小，在于标量的数量。 W_Q 的 512 维里包含了主谓、指代、修饰等所有信息，但经过点积全被压缩成一个标量。多头通过切分维度空间，让每个头在有限的

64 维里被迫专业化，因为容量小了，它没法同时照顾所有关系，只能把手头的 64 维做到极致。这就是「子空间专长」的来源。

这些专长不是人为指定的。是训练过程中自然分化出来的。你可以在训练后分析每个头的注意力模式，确实会发现头 1 偏爱主谓、头 2 偏爱指代，但没有人会在训练前告诉它们该学什么。

一个句子，8 个视角

句子：「昨天小明把他新买的手机忘在了图书馆」

头 1，主谓一致：「小明」→「忘」，确认动作主体。

头 2，指代消解：「他」→「小明」，不是随便一个路人。

头 3，修饰关系：「新买的」→「手机」，修饰语不是修饰「图书馆」。

头 4，局部邻接：「手机」←「忘」，相邻词形成动宾搭配。

头 5，长距离依赖：「昨天」（句首）→「忘」（句中），时间状语修饰。

头 6，介词结构：「在」→「图书馆」，地点标记。

头 7，标点/边界：句子首尾位置识别。

头 8，语义角色：「小明」施动、「手机」受动、「图书馆」地点。

\$W_O\$ 把这 8 个 64 维拼成的 512 维再揉一遍，做一个跨视角的融合，「主谓关系说小明是主语」+「指代关系确认他就是那个他」→最终确定「小明」是整个事件的施动者。

磨平一些信息差。

第 9 课：位置编码，模型怎么知道词顺序

Attention 公式里的 QK^T 是点积，点积是可交换的。 $Q_{\text{猫}} \cdot K_{\text{坐}} = Q_{\text{坐}} \cdot K_{\text{猫}}$ 。这意味着「猫追老鼠」和「老鼠追猫」在 Attention 眼里是完全一样的计算，算不出谁追谁。必须额外注入位置信息。

绝对位置编码：给每个座位贴门牌号

原版 Transformer 的做法：给位置 0 编一个 512 维向量，位置 1 编另一个 512 维向量，直接加到 Token 的 Embedding 上，然后正常做 Attention。位置编码用 sin 和 cos 函数生成：

$$\text{PE}(\text{pos}, 2i) = \sin\left(\frac{\text{pos}}{10000^{2i/d}}\right), \quad \text{PE}(\text{pos}, 2i+1) = \cos\left(\frac{\text{pos}}{10000^{2i/d}}\right)$$

偶数维度用 sin，奇数维度用 cos。 i 是维度索引， pos 是位置编号， d 是总维度 (512)。 $10000^{2i/d}$ 是一个随维度递减的频率，低维度变化慢（捕捉全局位置），高维度变化快（捕捉局部位置）。

致命伤是外推。训练时 pos 最大只到 511（序列长度 512），模型学会了「位置 511 的 cos 值大约是 0.62」。推理时序列长度 1024，位置 1023 的 cos 值是模型从未见过的数值，它不会处理。像乘法口诀表只背到 9×9 ，碰到 10×10 只能瞎猜。

相对位置编码：不算绝对位置，算谁离谁多远

换一种思路。不关心 token 在第几位，关心「两个 token 之间隔了多远」。距离 3 在句首还是句尾都一样。学一个距离到偏置的映射表：

$$\text{Attention_Score}(i, j) = Q_i \cdot K_j + \text{Bias}_{|i-j|}$$

$|i-j|=1$ 有一个可学习的偏置参数， $|i-j|=2$ 有另一个，一直到最大距离 k 为止。超过 k 的距离共享同一个偏置。

优势是平移不变，「猫坐垫子」不管在一篇文章的第 1 句还是第 100 句，内部距离关系一样。劣势是超过 k 的距离全被一视同仁（距离 500 和距离 5000 没有区别），而且参数表随

k 增长。

RoPE：不用加也不用学，直接旋转

当前最优方案。不把位置信息「加」到向量上，而是用旋转矩阵「乘」到 Q 和 K 上。

核心操作：把 Q 向量的每两个相邻维度看作平面上的一个点 (x, y)，然后用位置 m 乘以频率 θ 的角度旋转这个点。

$$R_m = \begin{bmatrix} \cos(m\theta) & -\sin(m\theta) \\ \sin(m\theta) & \cos(m\theta) \end{bmatrix}$$

不同维度对用不同的频率 θ 。 $\theta_i = 10000^{-2i/d}$ ，高维度转得快（对近距离敏感），低维度转得慢（对远距离敏感）。

旋转后计算 Attention 分数，奇迹发生了：

$$Q_m \cdot K_n = (R_m q)^T (R_n k) = q^T R_m^T R_n k = q^T R_{\{n-m\}} k$$

旋转矩阵的群性质： $R_m^T R_n = R_{\{n-m\}}$ 。两个旋转的复合等于角度差对应的旋转。所以内积的结果只依赖相对位置 (n-m)，跟绝对位置 m、n 无关。

位置 5 和位置 8 做 Attention，和位置 1005 和 1008 做 Attention，结果完全一样，都是旋转角 3。这就是外推强的根源：新位置只是多转几圈。

Llama、Qwen、DeepSeek 全都用 RoPE。有绝对编码的实现简洁（不需要额外参数表），同时有相对编码的外推能力。

RoPE 怎么工作的，两个词走一遍

句子只有两个词：「猫」（位置 0）、「坐」（位置 1）。 $d=4$ ， $\theta=[1.0, 0.5]$ 。

$$Q_{\text{猫}} = [0.8, 0.6, 0.3, 0.9]$$

位置 0 的旋转： - 维度 (0,1)：没有旋转，角度 = $0 \times 1.0 = 0 \rightarrow [0.8, 0.6]$ 不变 - 维度 (2,3)：没有旋转，角度 = $0 \times 0.5 = 0 \rightarrow [0.3, 0.9]$ 不变

$$Q_{\text{坐}} = [0.4, 0.2, 0.7, 0.5]$$

位置 1 的旋转： - 维度 (0,1)：旋转 $1 \times 1.0 = 1$ 弧度。 $\cos(1) \approx 0.54$, $\sin(1) \approx 0.84$ - 新值：
 $[0.4 \times 0.54 - 0.2 \times 0.84, 0.4 \times 0.84 + 0.2 \times 0.54] = [0.05, 0.44]$ - 维度 (2,3)：旋转 $1 \times 0.5 = 0.5$ 弧度。 $\cos(0.5) \approx 0.88$, $\sin(0.5) \approx 0.48$ - 新值： $[0.7 \times 0.88 - 0.5 \times 0.48, 0.7 \times 0.48 + 0.5 \times 0.88] = [0.38, 0.78]$

现在 $Q_猫 \cdot K_坐$ ，两个向量做内积。因为 $K_坐$ 也经过了同样的旋转，内积结果里自动包含了一个「旋转角 = $1 - 0 = 1$ 的相对位置信息」。如果句子变长到位置 100 和 101，旋转角还是 1，结果完全相同。

磨平一些信息差。

第 10 课：残差连接，梯度的高速公路

深层网络的最大敌人是梯度消失。反向传播时梯度从最后一层传回第一层，每层乘一个导数，通常小于 1。100 层累乘逼近零，浅层收不到更新信号。

残差连接：每层输出 $= x + F(x)$ 。原始输入 x 跳过硬处理直接送到加法节点。反向传播时 $dy/dx = I + dF/dx$ 。即使 dF/dx 接近零， I （单位矩阵）保底，梯度至少剩一个原封不动的拷贝。

这就是 Transformer 能堆 100 层以上的秘密。没有残差，12 层都训不动。

有残差 vs 没残差，梯度数值对比

假设一个 10 层 Transformer，每层的 FFN 子层导数平均是 0.8。

没有残差，纯链式法则：

第 10 层梯度：1.0
第 9 层： $1.0 \times 0.8 = 0.8$
第 8 层： $0.8 \times 0.8 = 0.64$
第 5 层： $0.8^5 = 0.33$
第 1 层： $0.8^9 = 0.13$

第 1 层收到的梯度只剩第 10 层的 13%。前面几层几乎学不到东西。

有残差， $dy/dx = I + dF/dx = 1.0 + 0.8 = 1.8$ ：

第 10 层梯度：1.0
第 9 层： $1.0 \times (1+0.8) = 1.8$
第 8 层： $1.8 \times (1+0.8) = 3.24$
第 5 层： $1.8^5 = 18.9$
第 1 层： $1.8^9 = 198.4$

梯度不仅没消失，反而在增大。这就是为什么残差网络每层至少保留 1.0 的基础梯度通路，不会死。

ResNet 2015 年提出时把 152 层网络训出来了，同期非残差网络 30 层就崩。Transformer 每层有两个残差：Attention 后加一个，FFN 后加一个。两条高速路，永不衰减。

磨平一些信息差。

第 11 课：LayerNorm，BN 在图像界称王，NLP 为什么不行

归一化就是把输入拉到标准分布上，减均值除以标准差，再乘 γ 加 β 。公式一样，区别在 μ 和 σ 从哪算。

数据形状是 (batch, seq_len, features)。BatchNorm 固定特征，跨所有样本和所有位置算均值。LayerNorm 固定一个样本的一个位置，在这个位置的特征内部算均值。

CV 用 BN。一张图 224×224 像素，batch=128，统计量巨大且稳定。

NLP 用 LN。句子不等长，有 PAD 填充位会污染 BN 统计量。NLP batch size 常很小，一个长句就占满显存。LN 不跨样本，batch=1 也正常工作，训练推理数学一致。

LN 在一个具体向量上走一遍

某层 Attention 输出一个 5 维向量：[2.0, -1.5, 3.2, -0.8, 1.1]

Step 1，算均值： $\mu = (2.0 - 1.5 + 3.2 - 0.8 + 1.1) / 5 = 4.0 / 5 = 0.8$

Step 2，算标准差： $\sigma = \sqrt{[(2.0-0.8)^2 + (-1.5-0.8)^2 + (3.2-0.8)^2 + (-0.8-0.8)^2 + (1.1-0.8)^2] / 5}$
 $= \sqrt{[(1.44 + 5.29 + 5.76 + 2.56 + 0.09) / 5]} = \sqrt{15.14/5} = \sqrt{3.03} \approx 1.74$

Step 3，归一化：每个元素做 $(x - 0.8) / 1.74$

[0.69, -1.32, 1.38, -0.92, 0.17]

均值变成了 0，标准差变成了 1。

Step 4， γ 和 β 放回来（假设 $\gamma=[1.2, 1.2, 1.2, 1.2, 1.2]$, $\beta=[0.1, 0.1, 0.1, 0.1, 0.1]$ ）：

[0.69×1.2+0.1, -1.32×1.2+0.1, ...] = [0.93, -1.48, 1.76, -1.00, 0.30]

γ 和 β 是 LayerNorm 保留的自由度。如果完全锁死在 [0,1] 范围，模型会丢失有用的幅值信息。它们让归一化后的向量保留一定的「个性」。

磨平一些信息差。

第 12 课：Pre-norm vs Post-norm，安检放哪的问题

两个公式。

Post-norm（原论文）： $y = \text{LayerNorm}(x + F(x))$ Pre-norm（现代）： $y = x + F(\text{LayerNorm}(x))$

区别：LayerNorm 放在加号前还是加号后。

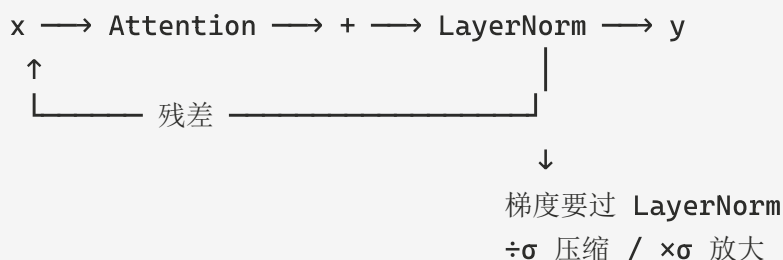
Post-norm 把残差和子层输出加起来再过 LayerNorm。梯度回传要经过除法，训练初期网络不稳，标准差上蹿下跳，梯度跟着波动。需要 Warmup：前几千步极小学习率等网络稳定。

Pre-norm 把 LayerNorm 挪到 $F(x)$ 前面。残差的平滑通道直接绕过整个 $F(\text{LayerNorm}(x))$ 块，梯度走这条不经过任何运算。 $dy/dx = I + dF(LN(x))/dx$ ， I 保底。不需要 Warmup，从第一步就能用正常学习率。

梯度回传路径对比

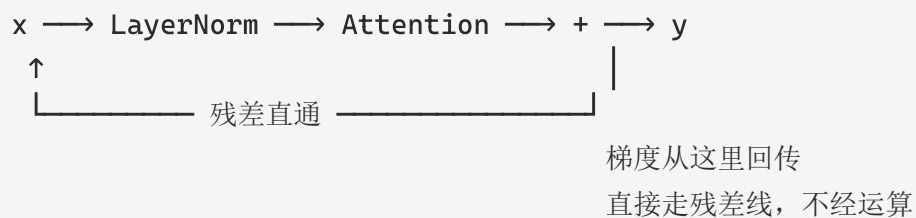
以一个 Attention 子层为例，输入 $x = [0.5, -0.3, 0.8]$ 。

Post-norm 路径：



梯度从 y 回传到 x ，必须穿过 LayerNorm 的除法运算。如果训练初期 $\sigma=0.1$ ，梯度被放大 10 倍，参数飞出去。如果 $\sigma=10.0$ ，梯度被压缩到 0.1 倍，参数原地踏步。这就是需要 Warmup 的根源。

Pre-norm 路径:



梯度从 y 回传时，残差线是纯直通。即使 Attention 输出归一化后的 σ 剧烈波动，残差线始终给出 $\partial y / \partial x = 1$ 的保底梯度。第一步就能正常训。

所有现代模型全用 Pre-norm。代价是对表达能力的微弱损失远小于工程稳定性的收益。Post-norm 已被历史淘汰。

磨平一些信息差。

第 13 课：两种 Mask，为什么模型必须戴眼镜

Padding Mask：句子不等长时用 PAD 补齐。PAD 位没有语义，不能参与 Attention。掩码位置设为 $-\infty$ ，过 softmax 后 $e^{(-\infty)}=0$ ，权重归零。

Sequence Mask（因果掩码）：下三角矩阵，对角线以下保留，以上全部屏蔽。Decoder 训练时不能偷看未来，推理时看不到未来，训练时看到了就会产生曝光偏差。

两个 mask 实际相加。被任一 mask 屏蔽的位置设为 $-\infty$ 。用 0 不行， $e^0=1$ ，被屏蔽的 token 仍会分到权重。只有 $-\infty$ 才能精确归零。

走一遍：训练「猫坐在垫子上」

输入序列含 PAD：[猫, 坐, 在, 垫子, 上, PAD, PAD]，长度补齐到 7。

Padding Mask 矩阵（1=允许, 0=屏蔽）：

```

猫 坐 在 垫 上 P  P
1 1 1 1 1 0 0  ← 猫看所有词
1 1 1 1 1 0 0  ← 坐看所有词
1 1 1 1 1 0 0  ← 在看所有词
1 1 1 1 1 0 0  ← 垫子看所有词
1 1 1 1 1 0 0  ← 上看所有词
1 1 1 1 1 0 0  ← PAD 看所有词（行也屏蔽）
1 1 1 1 1 0 0

```

PAD 的行和列全都屏蔽，既不查别人，也不被别人查。

Sequence Mask 矩阵：

```

猫 坐 在 垫 上
1 0 0 0 0  ← 猫只看自己

```

```

1 1 0 0 0 ← 坐看猫和自己
1 1 1 0 0 ← 在看前三
1 1 1 1 0 ← 垫子看前四
1 1 1 1 1 ← 上看所有（过去）

```

合并后的最终 Mask 矩阵（1=允许, 0=屏蔽）:

```

猫坐 在 垫 上 P  P
1 0 0 0 0 0 0 ← 猫只看自己
1 1 0 0 0 0 0 ← 坐看猫和自己
1 1 1 0 0 0 0 ← 在看前三个
1 1 1 1 0 0 0 ← 垫子看前四个
1 1 1 1 1 0 0 ← 上看全部历史
0 0 0 0 0 0 0 ← PAD 位全屏蔽
0 0 0 0 0 0 0

```

两个 Mask 做逻辑 AND：任一位置被屏蔽，最终就屏蔽。被屏蔽的位置在 Attention 分数里替换为 $-\infty$ ，Softmax 后权重归零。

Encoder 只加 Padding Mask（双向）。Decoder 自注意力加两个（因果 + Padding）。Decoder 的 Cross-Attention 只加 Padding Mask（K,V 来自 Encoder，不需要因果限制）。

磨平一些信息差。

第 14 课：参数 vs 数据，容器和内容

GPT-3 有 1750 亿参数，但只读了 3000 亿 Token。Chinchilla 只有 700 亿参数，却读了 1.4 万亿 Token。结果 Chinchilla 全面碾压。

参数是容量，数据是营养。大容量配不上足够的营养，就像 2 米的篮球运动员每天只吃一顿饭。

Chinchilla 论文给出经验公式：最优比例 $D \approx 20 \times N$ 。每 1 个参数喂约 20 个 Token。大多数早期大模型都是「大但没吃饱」，参数量巨大但数据量严重不匹配。

后续 Llama 用远超 Chinchilla 比例的数据训练，因为高质量数据可以多 epoch 重复使用，不严格遵守「只训一轮」假设。但 Chinchilla 的核心理念仍然成立：数据和参数要匹配增长。

Chinchilla 怎么算出 20:1 这个比例

固定算力预算 $C = 1000$ PFlops。Chinchilla 训了十几组 (N, D) 组合：

N (参数)	D (Token)	$C \approx 6ND$	Test Loss
2B	200B	$2.4e21$	2.85
4B	400B	$9.6e21$	2.62
7B	1.4T	$5.9e22$	2.45 ← Chinchilla 最优
10B	600B	$3.6e22$	2.68
70B	300B	$1.3e23$	2.91 ← Gopher 的配置

Chinchilla 70B + 1.4T tokens 用跟 Gopher 280B 一样的算力，Loss 低了将近 0.5。不是参数越多越好，而是在给定的 C 下找到 N 和 D 的最优平衡点。

公式： $N_{\text{opt}} \propto C^{0.5}$ ， $D_{\text{opt}} \propto C^{0.5}$ 。算力翻 4 倍，参数翻 2 倍、数据也翻 2 倍。两者对半增长，不是一边倒。

磨平一些信息差。

第 15 课：MLM vs CLM，两种截然不同的教法

MLM (Masked Language Model)：随机遮 15% 的词，模型根据前后文猜。BERT 的训练方式。优势是双向理解，猜被遮住的词需要同时看前后。劣势是只有 15% 的 Token 产生学习信号，且推理时没有 [MASK]，存在训练推理 mismatch。

CLM (Causal Language Model)：逐词预测下一个。GPT 的训练方式。每一个 Token 都是学习信号，训练和推理完全一致，都是预测下一个词。天然适合生成任务。劣势是单向，不能同时利用前后信息。

理解任务选 MLM。生成任务选 CLM。现代 LLM 全用 CLM，不是为了理解更好，是为了生成更强。

同一段文字，MLM 和 CLM 各能学出多少信号

训练语料：「今天天气很好，我和小明去公园散步」

CLM 方式，7 个 Token 产生 7 个预测任务：

今天 → 预测「天气」
今天 天气 → 预测「很好」
今天 天气 很好 → 预测「，」
...

7 个 Token，7 次学习机会。每个位置都在回答问题。

MLM 方式，随机遮 15%，产生 1 个预测任务：

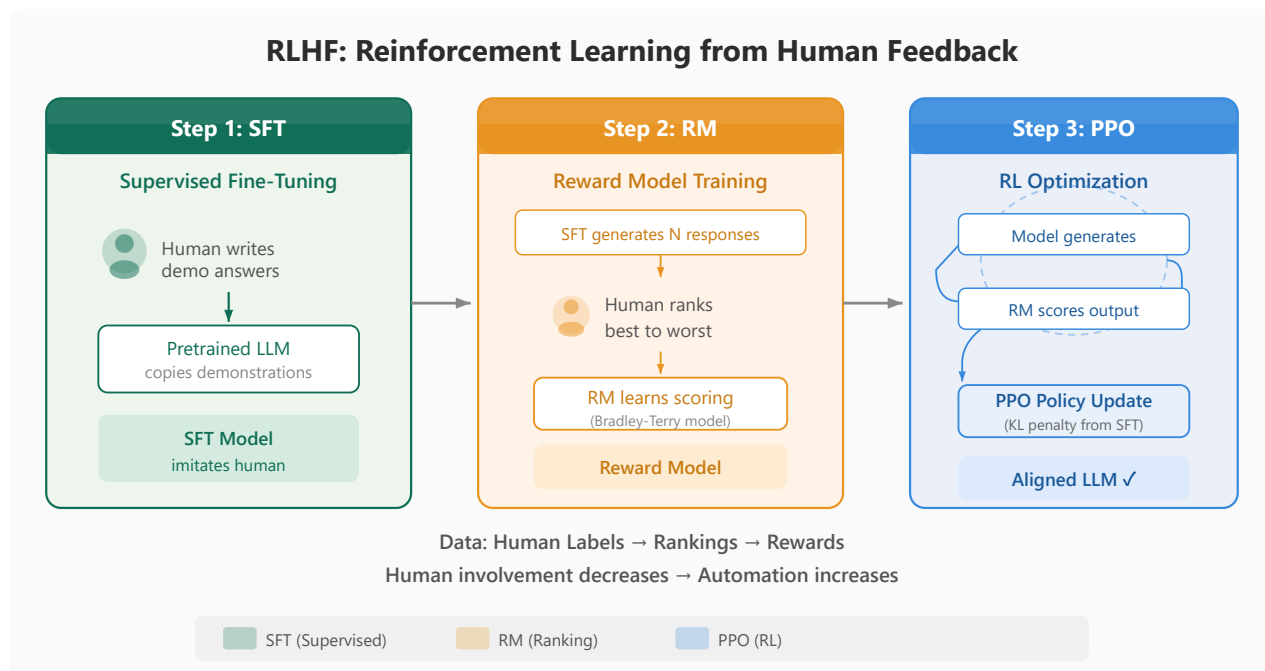
今天 [MASK] 很好，我 [MASK] 小明去公园散步
→ 预测「天气」和「和」

7 个 Token，只有 1 个 MASK token 产生信号（约 15%）。其余 6 个只提供上下文，不产生梯度。

同一段文字，CLM 产出的有效训练信号是 MLM 的约 6-7 倍。这就是 CLM 在互联网海量文本上的数据效率碾压 MLM 的根本原因。

磨平一些信息差。

第 16 课：RLHF 全流程，SFT → RM → PPO



RLHF 三阶段

SFT：人类写几千条「完美回答」，模型用交叉熵损失学着模仿。具体做法：标注员对着 prompt 手写「理想答案」，SFT 阶段就是让模型逐 token 预测标注员写的词。底层算法跟预训练一样（猜下一个词），但数据质量极高。学完能从「会说完整句子」变成「能写出及格水平的回答」。

RM：训练一个独立的打分模型。同一个 prompt，SFT 模型生成 4 个不同回答（A/B/C/D），人类标注员排序（ $A > B > C > D$ ），不打绝对分。为什么排序而非打分？因为人与人之间的绝对打分尺度差异太大，有人给 5 分极其慷慨，有人很苛刻。但所有人都能判断「A 比 B 好」。RM 用这个排序数据学习一个标量输出：给定 (prompt, response)，输出分数 r 。分数越高的回答越符合人类偏好。

PPO：策略模型（SFT 初始化）生成回答 → RM 打分 → 用强化学习算法更新策略参数 → 下一轮。反复循环直到模型稳定输出高质量回答。

PPO 到底在算什么

PPO 的核心操作是控制「新旧策略的差距」。策略 = 给定 prompt 下，模型输出各个 token 的概率分布。

定义概率比：
$$r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$$

分子是新策略在状态 s_t 下输出动作 a_t 的概率。分母是旧策略在同一状态输出同一动作的概率。 r_t 接近 1 表示新旧策略差不多，偏离 1 表示策略变化太大。

PPO 的损失函数：

$$\text{Loss} = -\min\left(r_t \cdot A_t, \text{clip}(r_t, 1-\epsilon, 1+\epsilon) \cdot A_t\right)$$

A_t 是优势函数，表示这个动作比平均水平好多少（RM 给出的奖励减去基准线）。

如果 $A_t > 0$ （这个动作好，应该多做），PPO 鼓励 r_t 变大，但不能超过 $1+\epsilon$ （通常 $\epsilon=0.2$ ）。超过之后梯度被截断，防止一次更新步子太大。

如果 $A_t < 0$ （这个动作差，应该少做），PPO 鼓励 r_t 变小，但不能低于 $1-\epsilon$ 。低于之后同样截断。

这就像限速：策略可以优化，但每次最多偏 20%。不能因为某个回答 RM 给了一次高分就把整个策略砸过去学它。

KL 散度约束和 Reward Hacking

PPO 的 Clip 机制控制单步更新幅度，但多轮累积后策略仍然可能慢慢「漂移」远离 SFT。所以额外加 KL 散度约束。

$$\text{Objective} = \mathbb{E}[r] - \beta \cdot \text{KL}(\pi_{\text{new}} \parallel \pi_{\text{SFT}})$$

KL 散度度量两个概率分布的差异。SFT 输出「苹果」的概率是 0.3，新策略输出是 0.9，KL 就会变大。 β 是惩罚系数（通常 0.01-0.1）。

一个真实案例：RM 从 Reddit 点赞数据训练，学到了「回答越长，RM 分越高」。PPO 阶段模型发现这个漏洞，开始在回答后面加礼貌废话，「希望对你有帮助，如有疑问请联系，祝你愉快」，RM 给高分但人类读了烦。

KL $\approx$ 0.05 (轻微偏离 SFT) $\rightarrow$ 正常优化

KL $\approx$ 0.5 (明显偏离 SFT) $\rightarrow \beta \times 0.5$ 抵消掉大部分 reward

KL $\approx$ 2.0 (翻墙) $\rightarrow$ 惩罚大于 reward, PPO 主动拉回来

这张 KL 网把模型锁在 SFT 的合理区域里优化，防它变成讨好 RM 的废话机器。RLHF 同时维护四个模型（策略模型、参考模型、奖励模型、价值模型），这就是它成本高的根本原因。

磨平一些信息差。

第 17 课：DPO 和 GRPO，更聪明的对齐方式

PPO 太贵。四个模型，两倍计算量，大量偏好标注，一堆超参。

DPO (Direct Preference Optimization)：直接干掉 RM 和 PPO。从偏好数据推一个跟 RLHF 等价的损失函数，一次梯度更新。只用一个模型，计算量等于 SFT。代价是对偏好数据质量高度敏感，没有 RM 的平滑缓冲。

GRPO (Group Relative Policy Optimization)：DeepSeek 的方案。也砍掉价值模型，改用一组回答的相对分数。同 prompt 生成 8 个回答，RM 打分，用均值和标准差算「相对优势」。KL 约束从 reward 里挪到 loss 里，执法更硬。

DPO 的损失函数，跟 PPO 到底差在哪

PPO 的目标：最大化 $E[r(x,y)] - \beta \times \text{KL}(\pi \parallel \pi_{\text{ref}})$

需要：策略模型生成 → RM 打分 → 计算 KL → 梯度更新。四个模型。

DPO 的数学推导发现，PPO 的最优策略有一个闭式解。把这个闭式解代回去，推导出一个只依赖偏好数据的损失函数：

$$L_{\text{DPO}} = -E[\log \sigma(\beta \times (\log \pi_{\theta}(y_w|x)/\pi_{\text{ref}}(y_w|x) - \log \pi_{\theta}(y_l|x)/\pi_{\text{ref}}(y_l|x)))]$$

不推导了，看它实际在做什么：

- y_w 是「更好的回答」， y_l 是「更差的回答」
- $\pi_{\theta}(y_w|x)/\pi_{\text{ref}}(y_w|x)$ ：新模型觉得好回答的概率 / 参考模型觉得好回答的概率
- 比值越大说明新模型比旧模型更喜欢好回答 → 奖励
- 优化方向：让新模型给好回答更高的相对概率

DPO 不需要 RM，不需要采样，不需要 PPO。你只需要准备好「(prompt, 好回答, 差回答)」三元组，一次梯度更新。数据质量和数量决定了上限。

GRPO 的做法更巧妙：同一个 prompt，生成 8 个回答。RM 给 8 个回答打分：[3.2, 4.1, 2.8, 3.9, 2.5, 4.4, 3.0, 3.7]。算均值 $\mu=3.45$ ，标准差 $\sigma=0.62$ 。每个回答的「相对优势」= $(\text{score} - \mu)/\sigma$ ：

回答 1： $(3.2-3.45)/0.62 = -0.40$ （低于平均）

回答 6： $(4.4-3.45)/0.62 = +1.53$ （明显高于平均）

优势为正的加分，为负的减分。不依赖绝对分数，RM 可能全打高了或低了，但相对的排序信息仍然有效。

磨平一些信息差。

第 18 课：LoRA，穷玩大模型的终极答案

全参微调 7B 模型要 56GB 显存。你只有 24GB。LoRA 只训 0.1% 的参数。

做法：原始权重 W 冻结不动。旁边加两个小矩阵 $B(d \times r)$ 和 $A(r \times d)$ 。 r 通常 8 或 16，远小于 $d(4096)$ 。 $B \times A$ 的结果尺寸跟 W 一样。推理时 $W' = W + B \times A$ ，速度跟原模型完全相同。

数学假设：模型适应新任务时的权重变化 ΔW 是低秩的，变化只发生在少数几个低维方向。实验验证成立。

QLoRA = LoRA + NF4 量化 + 双重量化。7B 模型只需 6-8GB 显存。RTX 3060 也能跑。

动手算一遍：一个 4×4 的权重矩阵， $r=2$

假设原始权重 W 是一个 4×4 矩阵（真实模型是 4096×4096 ，道理一样）。

W (4×4 , 冻结)

$$\begin{bmatrix} 0.5 & 0.3 & 0.1 & 0.8 \\ 0.2 & 0.7 & 0.4 & 0.1 \\ 0.9 & 0.2 & 0.6 & 0.3 \\ 0.1 & 0.5 & 0.2 & 0.7 \end{bmatrix}$$

B (4×2 , 可训)

$$\begin{bmatrix} 0.2 & 0.1 \\ 0.4 & -0.2 \\ 0.3 & 0.5 \\ -0.1 & 0.3 \end{bmatrix}$$

A (2×4 , 可训)

$$\begin{bmatrix} 0.3 & 0.5 & -0.1 & 0.2 \\ 0.1 & 0.6 & 0.3 & -0.1 \end{bmatrix}$$

Step 1: 算 $B \times A = \Delta W$

$B \times A =$

$$\begin{bmatrix} 0.2 \times 0.3 + 0.1 \times 0.1 & 0.2 \times 0.5 + 0.1 \times 0.6 & \dots \\ 0.4 \times 0.3 - 0.2 \times 0.1 & 0.4 \times 0.5 - 0.2 \times 0.6 & \dots \\ 0.3 \times 0.3 + 0.5 \times 0.1 & \dots & \dots \\ -0.1 \times 0.3 + 0.3 \times 0.1 & \dots & \dots \end{bmatrix}$$

$$\Delta W \approx \begin{bmatrix} 0.07 & 0.16 & -0.01 & 0.03 \\ 0.10 & 0.08 & -0.10 & 0.10 \\ 0.14 & 0.45 & 0.18 & 0.01 \\ 0.00 & 0.13 & 0.10 & -0.05 \end{bmatrix}$$

Step 2: 微调后的权重 $W' = W + \Delta W$

$$\begin{aligned} W'(0,0) &= 0.5 + 0.07 = 0.57 \\ W'(0,1) &= 0.3 + 0.16 = 0.46 \\ &\dots \end{aligned}$$

只改了极小一部分。大多数值几乎不变，但这几个低维方向上的调整足够适配新任务。

参数的差距： - 全参微调要更新 W 的全部 16 个值 - LoRA 只更新 $B(8\text{个}) + A(8\text{个}) = 16$ 个值？不对，这里 4×4 太小，看不出优势。换 4096×4096 ： - 全参更新 16,777,216 个值 - LoRA ($r=8$) 更新 $4096 \times 8 + 8 \times 4096 = 65,536$ 个值 - 差了 256 倍。这就是 LoRA 省显存的原因。

训练完把 ΔW 合入 W ，推理零开销。

磨平一些信息差。

第 19 课：训练硬核补充，过拟合、梯度、Warmup、ZeRO

Warmup：训练初期用极小学习率，几百到几千步后加到正常值。Post-norm 时代必须，Pre-norm 后必要性大降但实践者仍会加极短的 Warmup 做保险。

过拟合：训练 Loss 降，验证 Loss 升。解法，正则化（Dropout、权重衰减）、更多数据、早停。PEFT 天然防过拟合。

梯度爆炸：梯度过大，参数跳飞。解法，梯度裁剪，范数超限就等比缩放到阈值（默认 1.0）。

梯度消失：梯度过小，参数不动。解法，残差连接 + 好的初始化。Transformer 有残差，问题大幅缓解。

ZeRO：分布式训练的内存优化。标准数据并行每张卡都存完整优化器状态，8 张卡就 8 份冗余。ZeRO-1 优化器状态分片。ZeRO-2 梯度分片。ZeRO-3 参数也分片。

ZeRO 三种等级，显存怎么省出来的

以一个 7B 模型（FP16，14GB 参数）为例，Adam 优化器在 8 卡上的显存占用：

组件	单卡占用	8 卡总计	说明
参数	14 GB	$14\text{ GB} \times 8 = 112\text{ GB}$	每卡一份完整模型
梯度	14 GB	112 GB	每卡一份完整梯度
优化器状态	28 GB (×2)	224 GB	Adam 一阶+二阶矩，参数两倍
总计	56 GB	448 GB	96% 浪费在了冗余

ZeRO-1：优化器状态分片到 8 卡，每卡只存 $28/8 = 3.5\text{ GB}$ 。单卡显存 $56 \rightarrow 31.5\text{ GB}$ 。节省 44%。

ZeRO-2：梯度也分片，all-reduce 时只传自己负责的那片。单卡 31.5 → 17.5 GB。节省 69%。

ZeRO-3：参数也分片。前传时从别的卡拉参数，用完丢。单卡 17.5 → 3.5 GB。节省 94%。

一张 40GB A100 训不了 7B 全参微调（需要 56GB）。加 ZeRO-3，3.5 GB 就能训。

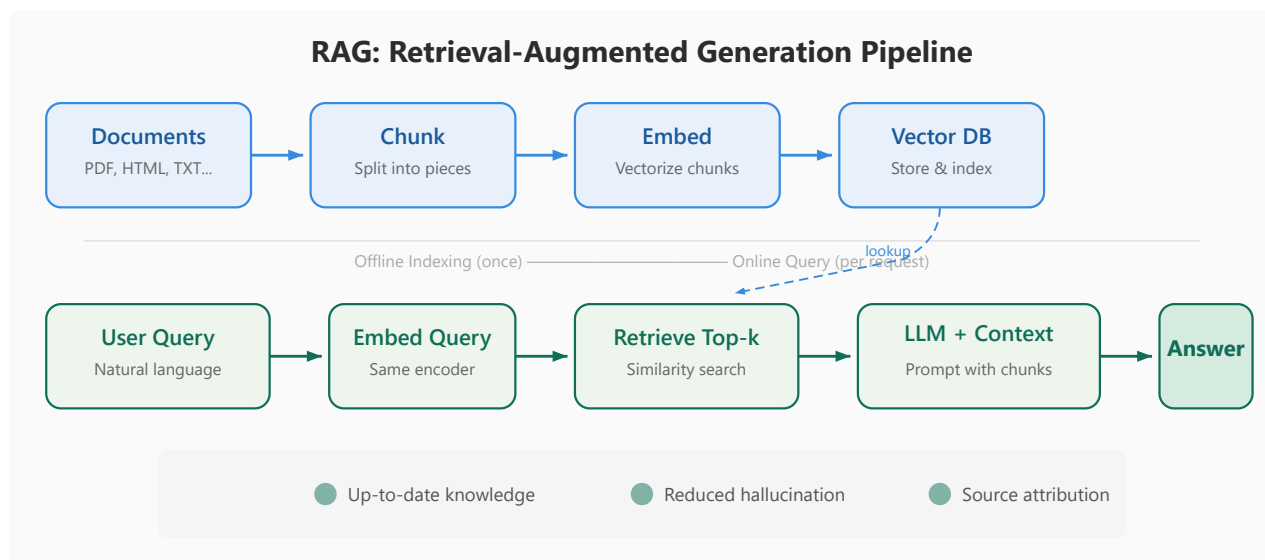
磨平一些信息差。

第 20 课：RAG，让模型会查资料

大模型会编造不存在的 API、不存在的书名、不存在的历史事件。幻觉。

RAG 不给模型凭记忆回答。给它一本参考书，让它翻完再说。

五步。切分：文档切成 200-500 Token 小块。块太大检索不准，太小上下文不够。建索引：每块用 embedding 模型转向量，存进向量数据库。查询：用户问题转向量，检索最接近的 Top-K 块。混合检索：向量检索 + BM25 关键词检索，拼起来重排序，语义相近和精确匹配兼顾。生成：拼成 Prompt 「根据以下资料回答。资料：[检索结果]。问题：[用户问题]」发给大模型。



RAG 流水线

RAG 怎么干掉一次幻觉，完整链路走一遍

用户问：「公司去年 Q3 的销售额是多少？」

Step 1 — 切分：把公司年报切成 300 Token 的小块。财务部分独占 5-6 块，每块标页码和段落号。

Step 2 — embedding：每块过 text-embedding-3-small 转向量，存入 Milvus。

Step 3 — 查询向量化：「公司去年 Q3 的销售额是多少」→ [0.23, -0.45, 0.67, ...] 1536 维。

Step 4 — 检索：向量库返回 Top-5 最相似块。其中 Top-1 匹配到「2025 年第三季度，公司实现销售收入 3.2 亿元」，余弦相似度 0.94。BM25 关键词「Q3」「销售额」也命中同一段。

Step 5 — 生成 Prompt：

根据以下资料回答，资料来源于公司 2025 年年报。

资料 1（相似度 0.94）：「2025 年第三季度，公司实现销售收入 3.2 亿元，同比增长 18.5%

问题：公司去年 Q3 的销售额是多少？

Step 6 — 模型输出：「根据公司 2025 年年报，去年第三季度销售额为 3.2 亿元。」

没有 RAG，模型可能猜「约 2.5 亿」或者编一个数字。有 RAG，它从资料里精确引用。幻觉风险从「可能编」降到了「除非检索失败否则不编」。

磨平一些信息差。

第 21 课：推理优化全家桶

训练可以并行，推理必须串行，每次只生成一个 Token。GPU 空着等。

KV Cache：已算过的 K 和 V 缓存起来，下一个 Token 只算新的 K、V。省计算，但 Cache 本身占显存，序列越长越大。

vLLM + PagedAttention：把 KV Cache 像操作系统内存分页一样切块，需要时分配用完回收。相比传统预分配连续大块，碎片化大幅减少，吞吐翻几倍。

量化：推理时 FP16→INT8 显存减半速度翻倍。INT4 再减半。

FlashAttention：不把完整的 $n \times n$ 矩阵写进 HBM，分块算，online softmax 保证数值正确。

投机解码：小模型快速生成几个候选 Token，大模型一次性验证。对的多全收，错的扔掉重来。

KV Cache 到底省了多少计算，一个序列走一遍

序列「今天天气很好」已生成 5 个 Token，下一步生成「，」。

没有 KV Cache：

每一步都要重新算全部 K 和 V：

Token 1：算 K_1 , V_1 (1 次计算)
Token 2：算 K_1 , V_1 , K_2 , V_2 (2 次)
Token 3：算 $K_1 \dots K_3$, $V_1 \dots V_3$ (3 次)
Token 4：算 $K_1 \dots K_4$, $V_1 \dots V_4$ (4 次)
Token 5：算 $K_1 \dots K_5$, $V_1 \dots V_5$ (5 次)

总共 15 次 KV 计算。实际场景序列长度 $n=4000$ ，总计算量 $O(n^2) \approx 1600$ 万次乘法。

有 KV Cache：

Token 1: 算 K_1 , V_1 , 缓存
Token 2: 算 K_2 , V_2 , 缓存 (读 K_1 , V_1)
Token 3: 算 K_3 , V_3 , 缓存 (读 K_1-K_2 , V_1-V_2)
...

每个新 Token 只算自己的 KV，前面已算的全部从缓存读。序列长度 $n=4000$ ，计算量 $O(n) \approx 4000$ 次。加速约 4000 倍。代价是 KV Cache 占用显存：每层每头存 K 和 V，以 GPT-3 175B 为例约 96 层 $\times$ 96 头 $\times 2 \times (K+V)$ ， $n=2048$ 时约 2.7 GB。

磨平一些信息差。

第 22 课：AI Agent，让模型会干活

Chatbot 是被动回答，你问一句它答一句，答完就忘。Agent 是主动完成任务，感知环境、做决策、调工具、看结果、调整下一步，循环直到目标达成。

Function Calling：模型的「遥控器」

Agent 最基础的能力是调用外部工具。机制叫 Function Calling：

1. 提前定义好工具清单（函数名、参数格式、功能描述），写在 system prompt 里
2. 用户输入后，模型决定要不要调工具。要调的话，输出一串 JSON 描述「调哪个函数、传什么参数」
3. 外部框架截获这串 JSON，执行真正的函数调用（查数据库、调 API、读文件）
4. 把返回结果追加到对话历史里，喂回给模型
5. 模型基于工具返回结果继续推理或输出最终回答

模型自己不能上网、不能查数据库、不能发微信。它只会生成 JSON。是外部框架在替它执行。这种设计让模型的能力可以无限外挂，加一个新工具只需要在清单里多写一行描述。

ReAct：Agent 的决策循环

ReAct = Reasoning + Acting。最主流的 Agent 决策模式。不是「想好了再动手」，而是「想一步、做一步、看结果、再想下一步」。

一个完整的 ReAct 循环：

Thought（思考）：我现在知道了什么，下一步该做什么
Action（行动）：调用工具
Observation（观察）：工具返回了什么
→ 回到 **Thought**，基于新信息继续

这套循环让 Agent 具备了两个 Chatbot 没有的能力。多步推理：复杂任务不能一步完成，Agent 自己拆成若干步，每一步基于上一步的结果调整。错误恢复：工具返回错误时，Agent 换参数重试或换工具，不会直接输出一个错的最终答案。

一个 Agent 的实际执行过程

任务：「帮我查一下北京明天天气，如果下雨就给我的微信发条提醒」

Thought 1：「先查天气。」 Action 1：生成 Function Call

```
{"function": "get_weather", "parameters": {"city": "北京", "date": "2026-07"}}
```

Observation 1: `{"weather": "中雨", "temp": "26°C"}`

Thought 2：「中雨，需要提醒。」 Action 2：生成 Function Call

```
{"function": "send_wechat", "parameters": {"to": "self", "message": "明天北"}}
```

Observation 2: `{"status": "sent"}`

Thought 3：「搞定了，回复用户。」 Final Answer：「明天北京中雨 26°C，微信提醒已发送。」

Chatbot 只能输出「明天北京可能下雨」。Agent 真的调了 API 查天气，并且真的发了微信。

单 Agent vs 多 Agent

单 Agent 手里有所有工具，自己决策全过程。简单直接，适合任务边界清晰、工具不超过 10 个的场景。能力受限于单个模型的推理深度。

多 Agent 各司其职。一个检索 Agent 从数据库拉数据，一个分析 Agent 做结论，一个写作 Agent 出报告。Agent 之间通过消息传递信息。复杂度指数增长，通信死锁、等待链、谁来仲裁，需要在架构层面做超时和容错。

Google 的 A2A 协议就是为多 Agent 互操作设计的。第 23 课详讲。

磨平一些信息差。

第 23 课：MCP / A2A / Function Call，让模型会叫外援

Function Call：模型输出 JSON 描述调什么函数传什么参。外部框架截获执行后结果喂回。极简，但一次性，没有持久化工具管理。

MCP (Model Context Protocol)：Anthropic 2024 年发布。客户端-服务器架构。AI 宿主是客户端，每个工具封装成 MCP 服务器。服务器暴露统一接口，列出资源、描述参数、处理请求。一次开发，所有支持 MCP 的 AI 应用都能用。社区已有数百个现成服务器，Google Drive、GitHub、PostgreSQL。

A2A (Agent-to-Agent)：Google 2024 年发布。不是模型↔工具，是 Agent↔Agent。跨组织互操作，采购 Agent 跟供应商库存 Agent 确认交货时间。定义了发现、握手、任务分配、结果回传的标准流程。

Skill：预定义的 Prompt + 调用规范。MCP 是通用工具层，Skill 是场景化编排。两者互补。

MCP 连接数据库的实际过程

假设 Claude 要通过 MCP 查 PostgreSQL 里的订单表。

1. 启动时：MCP 客户端（Claude 宿主）连接 PostgreSQL MCP 服务器。服务器暴露能力清单：

```
{
  "tools": [
    {"name": "query", "description": "执行 SQL 查询", "parameters": {"sql": {
      "name": "list_tables", "description": "列出所有表"
    }}}
  ]
}
```

2. 用户提问：「查一下 user_id=1088 昨天的订单总金额」

3. 模型生成 Function Call:

```
{  
  "tool": "query",  
  "parameters": {  
    "sql": "SELECT SUM(amount) FROM orders WHERE user_id = 1088 AND date =  
  }  
}
```

4. MCP 服务器执行：连接数据库，跑 SQL，返回 `[{"sum": 456.80}]`

5. 模型回答：「用户 1088 昨天的订单总金额为 456.80 元。」

整个过程，模型不需要知道数据库地址、账号密码、表结构。MCP 服务器做了封装。换一个 MySQL 服务器，只要它实现了相同的 MCP 接口，模型代码零改动。

磨平一些信息差。

第 24.5 课：多模态/VLM 入门，模型怎么看图

到目前为止讲的都是文本模型。但 2025 年之后，面试里越来越多问多模态。GPT-4V、Gemini、Qwen-VL 都能看图了。怎么做到的？

CLIP：图文的翻译官

2021 年 OpenAI 发布了 CLIP，核心操作极其简单：用对比学习让图片和文字在同一个向量空间里对齐。

训练数据是 4 亿对（图片，文字描述）。batch 里混着放，目标是让配对的图文向量距离最近，不配对的推远。训练完后，一张猫的图片拉成的向量，和「一只猫」这行文字拉成的向量，在 512 维空间里几乎重叠。和「一辆汽车」的向量则隔得很远。

CLIP 的结构：一个视觉编码器把图片压缩成 512 维向量，一个文本编码器把文字也压缩成 512 维向量。两个向量做余弦相似度。它是目前所有 VLM 的地基。

VLM 怎么把图喂给 LLM

标准架构分三步。

第一步，视觉编码器（通常是 CLIP 的 ViT 或 SigLIP）把一张图切成若干 patch（比如 16×16 像素的小块），每个 patch 变成一个向量。一张 224×224 的图变成 196 个 patch 向量。

第二步，投影层（一个线性层或轻量 MLP）把这 196 个视觉 patch 向量投影到跟 LLM 的 Token Embedding 同一个维度。比如 LLM 的 Embedding 是 4096 维，投影层就把 ViT 输出的视觉向量全映射到 4096 维。

第三步，拼到 LLM 输入里。正常的文本 Token 是 [猫，坐在，垫子上]。加上图片之后变成 [img_patch_1, img_patch_2, ..., img_patch_196, 猫，坐在，垫子上]。LLM 通过自注意力同时看到视觉信息和文本信息，自然能把「图片里那只猫」跟「猫这个字」对齐。

LLaVA 就是这个架构。视觉编码器冻结不动（CLIP 已经训好了），LLM 冻结不动（已有语言能力），只训练中间那层投影层。成本极低，几小时就训完。

VLM 也会幻觉，而且更隐蔽

纯文本幻觉：模型不知道某个事实，编了。VLM 幻觉：模型看错了图片里的东西，然后基于错误的视觉理解做推理。

一张模糊的停车标志图，视觉编码器看成了限速标志。LLM 拿到的是一个「限速标志」的向量，然后一本正经地推理这张图的交通含义。从 LLM 的视角看，它没编造，它收到的视觉输入本身就是错的。

这也是为什么 VLM 对光照、角度、遮挡极其敏感。视觉编码器的感受野有限，远处小物体、被遮挡物体、反光区域都是幻觉重灾区。

面试三问 + 怎么答

「VLM 和纯 LLM 在架构上最大的区别是什么？」— 多了一个视觉编码器 + 投影层。LLM 本身没变。训练时只训投影层（轻量方案）或者把 LLM 也解冻（重量方案）。

「图文怎么对齐的？」— CLIP 的对比学习。核心 loss 让配对的图文向量互信息最大化。对齐质量直接决定 VLM 的上限。

「VLM 的推理成本为什么比 LLM 高？」— 图的 patch 数量远大于文本 Token。224×224 的图 = 196 个 patch = 196 个 Token 的视觉前缀。加上原始文本 Token，序列长度至少翻倍。自注意力的 $O(n^2)$ 随着 Token 翻倍，计算量翻 4 倍。

磨平一些信息差。

第 24 课：前沿架构，MoE、GQA、DeepSeek 怎么玩的

MoE (Mixture of Experts)：把密集的 FFN 拆成多个「专家」小矩阵。每个 Token 经由路由网络只激活 Top-2 专家。参数量暴增但激活量不暴增。DeepSeek-V3 总参数 1.6T，每次推理只激活 49B。负载均衡通过辅助损失保证各专家使用率均等。

GQA (Grouped Query Attention)：K 和 V 分成 G 组，Q 保持全头，组内共享 KV。MHA (全头) 精度最高但 KV Cache 大。MQA (K/V=1 头) 最省但信息瓶颈。GQA 取折中。Llama 2、Qwen 用 GQA。

DeepSeek-V3 整合了 MoE + GQA + MLA (Multi-head Latent Attention，KV 投影到极小的潜在空间，128K 长序列 KV Cache 压至传统方案 1/10)。开源模型顶尖。

MoE 怎么做到「参数巨大但推理不贵」

传统密集模型：一个 7B 模型，每个 Token 都要经过全部 7B 参数。计算量 = 7B。

DeepSeek-V3：256 个专家（每个约 6.4B），每次只激活 Top-8。

一个 Token 穿过 MoE 层的计算：

Token 「猫」→ 路由网络打分 →

专家 17：0.83 (激活)

专家 89：0.72 (激活)

专家 3：0.15 (忽略)

...

专家 201：0.08 (忽略)

输出 = $0.83 \times \text{Expert17}(\text{Token}) + 0.72 \times \text{Expert89}(\text{Token})$

256 个专家，只跑了 8 个。参数量 1.6T，每次推理的计算量只相当于激活参数 49B。这就是 MoE 的精髓：大模型的容量，小模型的速度。

负载均衡怎么保证：如果所有 Token 都选专家 17 和 89，其他 254 个专家白训了。辅助损失函数惩罚专家使用不均：

$$L_{\text{balance}} = \alpha \times \sum (\text{每个专家的平均选择概率} - 1/N_{\text{experts}})^2$$

所有专家使用率强制均匀分布。训练几亿 Token 后，256 个专家自然分化出各自的领域。

磨平一些信息差。

第 25 课：大厂真题精讲（上）

五道真题，每题覆盖一个核心面试方向。

字节：PEFT 方案怎么选。 通用 LoRA ($r=8-16$)。显存紧 QLoRA。精度控 Adapter。简单分类 Prompt Tuning。 r 值看任务复杂度，简单情感 $r=4$ ，复杂推理 $r=32$ 。LoRA 训在 Q、V 上效果最好。QLoRA 实测显存比理论高 10%-20%。

腾讯：RAG 系统怎么设计。 切分 200-500 Token→BGE/text-embedding-3 建索引→向量+BM25 混合召回→Cross-Encoder 重排→拼 Prompt 喂 LLM。Chunk size 从 300 开始试，看检索召回率。ChromaDB 做原型，Milvus 上生产。RAGAS 做评估。

阿里：偏差和方差。 训练&验证 Loss 都高→偏差高（欠拟合）→加数据加参数加步数。训练低验证高→方差高（过拟合）→正则化。PEFT 天然低方差。全参微调数据少时方差极高。Debug 看两 Loss 的 gap。

美团：Agent 怎么拆。 感知层（用户输入+工具返回+上下文）、决策层（ReAct：思考-行动-观察）、执行层（Function Calling）、记忆层（短期+长期）。单 Agent vs 多 Agent 看任务边界。工具调错→三层防御：参数校验、异常重试（N 次换参换工具）、回退兜底。

小红书：MoE 怎么玩更稳。 负载均衡加辅助 loss。All-to-All 优化通信。保守学习率配频繁负载检查。

模拟面试：完整回答一道 RAG 题

面试官：「给你一个包含 500 篇技术文档的内网知识库，设计一个 RAG 问答系统。」

你的回答：

「我分五步来。」

切分，先试 300 Token 的 chunk，overlap 设 50 Token。然后看实际文档类型调，如果大部分是 API 文档，结构性强，可以切得更短（200 Token）。如果是综述类长文，拉到 500。

Embedding，用 BGE-large-zh 或 text-embedding-3-large，1536 维。如果文档涉及大量代码，考虑训练 domain-specific 的 embedding 模型。

检索，向量检索做语义召回，BM25 做关键词精确匹配。Top-20 召回来之后，用 Cross-Encoder 做重排序，取 Top-5 送入 LLM。这个两步检索能兼顾语义和精确。

生成 Prompt，给每个 chunk 标上来源、相似度分数，要求模型「如资料中没有相关信息，请直接说明不知道，不要编造」。这一句能把幻觉率降一截。

评估闭环，上线后用 RAGAS 框架定期抽检，看检索召回率和答案准确率。用户反馈里搜集 badcase，回到切分策略上迭代 chunk size。」

面试官大概率会在 chunk size 怎么调、混合检索的实现细节、Cross-Encoder 的性能开销上追着问。提前准备好这三个点的具体回答。

磨平一些信息差。

第 26 课：大厂真题精讲（下） + 系统设计 + 项目讲述

最后五道真题 + 一道系统设计大题 + 怎么在面试里讲自己的项目。

京东：模型上线评估闭环。 部署（vLLM/TGI+量化+KV Cache），监控（延迟 P50/P99、吞吐、错误率、显存），评估（A/B 测试+离线 benchmark），反馈（badcase 收集→标注→加入训练集→月/季度更新）。延迟飙升排查：GPU 利用率掉→等 IO。KV Cache 超限→序列太长。batch 被拖累→超长序列塞在里面。

百度：SFT 和对齐原理选路线。 SFT 是模仿人类写的好回答。对齐是偏好学习。首期 SFT，二期上对齐。预算足+安全要求高→RLHF。预算紧→DPO，不训 RM。生成任务为主→GRPO，省价值模型。

滴滴：Transformer 为什么取代 RNN。 并行处理取代串行。自注意力取代长距离梯度消失。残差连接让我们堆 100+ 层。

开放题：挑一个深入研究的模型。 挑你真懂的。比如 DeepSeek-V3 的 MoE 降推理成本、GQA 优化 KV Cache、MLA 压缩潜在空间做到 128K 长上下文。这是热情测试，不是知识测试。

系统设计：大模型项目架构。 七模块：数据层（清洗+标注管线）、模型层（选基座+微调策略）、训练层（分布式+ZeRO+混合精度）、推理引擎层（vLLM+量化+KV Cache）、应用层（Agent+RAG+Prompt 管理）、监控层（延迟+吞吐+badcase）、安全层（输入输出护栏+越狱检测）。每模块挑一个你真做过的展开五句。

讲项目，正面 vs 反面示范

反面（面试官秒睡版）：

「我做了一个智能客服系统，用了大模型和 RAG。效果挺好的，准确率提高了 30%。」

什么都没说，什么模型、什么数据、什么挑战、什么指标、什么基准线。

正面（面试官追问版）：

「我在 XX 公司做了一套智能客服系统，处理电商的退货咨询。之前是规则引擎，覆盖率只有 40%，剩下的转人工。

我选了 Qwen2.5-7B 做基座。数据是自己整理的 8000 条客服对话，加上 2000 条人工标注。微调用 LoRA， $r=16$ ，在 Q、V 上加法，两张 A100 跑了两小时。

RAG 部分是最大的坑。退货政策经常更新，实时性很重要。我做了混合检索，向量检索负责语义相近的询问，BM25 做关键条款号的精确匹配。检索失败的时候模型会编退货政策，非常危险。我加了强制引用，要求回答必须标注出处段落号。没引用就用兜底话术转人工。

上线效果：自动解决率从 40% 提到 78%，每天少转 400 通人工。最大遗留问题是多轮对话的上下文理解，用户说「那个东西还是不对」这种指代消解经常失败。下一版考虑加对话 memory。」

为什么这个回答好：

- 有具体的基座模型名（Qwen2.5-7B）和微调配置（LoRA $r=16$ ）
- 有真实数据量（8000+2000 条）和硬件信息（两张 A100）
- 讲了最大坑（模型编退货政策）和解决方案（强制引用+兜底转人工）
- 有量化指标（40%→78%，400 通/天）
- 坦诚说了遗留问题（多轮指代），显得真实

Token 是零件。预训练是学拼法。Transformer 是图纸。SFT 和 RLHF 是请师父精进。LoRA 是换零件不换整盒。RAG 是给参考书。Agent 是让乐高自己动起来。

二十六课串成了一条线。你能用自己的话讲清楚 Attention 是什么，LoRA 为什么好使，模型为什么必须戴 Mask，这本书就没白写。

磨平一些信息差。

《AI 大模型不难》—— 开源电子书
灵感来自鸢尾花书的视觉密度和数字生命卡兹克的叙事温度
[GitHub](#)